

OBC SMART

Logistics Courier Marketplace Platform

SOFTWARE REQUIREMENTS SPECIFICATION

SRS Document — Version 1.0

Project Name:	OBC Smart Flutter Application
Platform:	Android & iOS (Flutter)
Prepared For:	BFOBC / OBC Flutter Project
Document Status:	Final
Date:	May 14, 2026
Version:	1.0.0

TABLE OF CONTENTS

1.	Introduction	3
2.	Overall Description	4
3.	System Architecture & Technology Stack	5
4.	User Roles & Actors	6
5.	Functional Requirements — Authentication	7
6.	Functional Requirements — Broker Module	8
7.	Functional Requirements — Courier Module	10
8.	Functional Requirements — Chat & Communication	12
9.	Functional Requirements — Notifications	13
10.	Functional Requirements — Maps & Location	14
11.	Functional Requirements — Admin Panel	15
12.	Non-Functional Requirements	16
13.	Data Models	18
14.	External Interfaces	20
15.	Security Requirements	21
16.	Constraints & Assumptions	22
17.	Glossary	23

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **OBC Smart** mobile application — a Flutter-based logistics courier marketplace platform. This document is intended for developers, QA engineers, project managers, and stakeholders involved in the development and deployment of the OBC Smart system.

1.2 Scope

OBC Smart is a dual-role mobile application for Android and iOS that connects **Brokers** (companies or individuals who post logistics job opportunities) with **Couriers** (individuals who fulfill those jobs). The platform enables real-time job discovery, bidding, tracking, and communication within the aviation logistics sector.

- Real-time courier marketplace with map-based discovery
- Dual-role user system: Broker and Courier
- Job posting, milestone tracking, and status management
- Empty aircraft leg management and discovery
- In-app real-time chat and file sharing
- Firebase-backed authentication and cloud data storage
- Push notifications for job updates and messages
- Rating and review system for quality assurance
- Admin panel for user management
- Airport database with 10,000+ records

1.3 Document Conventions

Term	Description
FR-XXX	Functional Requirement identifier
NFR-XXX	Non-Functional Requirement identifier
High	Must be implemented for MVP
Medium	Important but can be deferred
Low	Nice-to-have feature
Broker	User who posts job opportunities
Courier	User who fulfills job opportunities

1.4 References

- Flutter SDK Documentation — flutter.dev
- Firebase Documentation — firebase.google.com

- Google Maps Flutter Plugin — pub.dev/packages/google_maps_flutter
- OBC Flutter Project Repository — github.com/BFOBC/OBC_Flutter_Project
- Firebase Cloud Messaging (FCM) — firebase.google.com/docs/cloud-messaging

2. Overall Description

2.1 Product Perspective

OBC Smart operates as a standalone mobile application backed by Google Firebase cloud services. It interfaces with Google Maps for location visualization, Firebase for data persistence and authentication, and FCM for push notifications. The app targets the aviation courier logistics market, enabling brokers to find couriers and manage cargo transport missions.

2.2 Product Functions (High-Level)

Module	Key Functions
Authentication	Register, Login, Role Selection, Profile Completion
Broker Dashboard	Post Jobs, Search Couriers, Manage Missions, Track Milestones
Courier Dashboard	Browse Jobs, View Flight Opportunities, Accept Missions, Complete Tasks
Maps & Location	Real-time GPS tracking, Google Maps integration, Courier discovery
Empty Legs	Post/Browse empty aircraft legs, Request joining flights
Chat	Real-time messaging, File/Media sharing, Conversation history
Notifications	Push notifications, In-app alerts, Background notifications
Ratings	Rate couriers/brokers, View ratings, Submit reviews
Admin Panel	User management, Account status control, Statistics
Airport DB	Search 10,000+ airports, IATA code lookup, Coordinate data

2.3 User Classes and Characteristics

User Class	Description	Technical Level
Broker	Companies/individuals posting courier jobs. Manage missions, miles, cargo, payments	Basic-Intermediate
Courier	Professionals fulfilling logistics tasks. Browse jobs, track progress.	Basic
Admin	Platform administrators managing user accounts and system health	Advanced
Guest	Unauthenticated users with access only to login/registration screens	Basic

3. System Architecture & Technology Stack

3.1 Architectural Overview

OBC Smart follows a **Client-Server architecture** with a Flutter-based mobile client and Firebase as the cloud backend. The app employs the **Provider pattern** for state management and a service layer (FirestoreService, AirportService, NotificationService) for backend communication. Local data is cached using SQLite for offline airport lookups and SharedPreferences for user preferences.

3.2 Technology Stack

Category	Technology	Version	Purpose
UI Framework	Flutter	≥3.1.5	Cross-platform mobile UI
Language	Dart	Latest	Application logic
State Management	Provider	6.1.2	App-wide state management
Authentication	Firebase Auth	5.3.3	User sign-in/sign-up
Primary Database	Cloud Firestore	5.5.0	NoSQL cloud database
Realtime Sync	Firebase Realtime DB	11.1.6	Live data updates
File Storage	Firebase Storage	12.3.6	Documents & media
Push Notifications	Firebase Messaging	15.2.5	FCM push notifications
Local DB	SQLite (sqflite)	2.3.3+1	Airport data caching
Maps	Google Maps Flutter	2.3.0	Map display & markers
GPS	Geolocator	10.1.0	Real-time location tracking
Geocoding	Geocoding	2.1.0	Address ↔ coordinates
HTTP Client	Dio	5.3.2	API requests with interceptors
Background Tasks	WorkManager	0.5.1	Scheduled background jobs
Local Notifications	Flutter Local Notifications	17.1.2	In-app notification display
Charts	Fl Chart + Pie Chart	0.69.0 / 5.4.0	Analytics visualization
Animations	Lottie	2.6.0	Splash & UI animations
Image Loading	Cached Network Image	3.3.1	Cached image rendering
Connectivity	Connectivity Plus	5.0.2	Network status monitoring

3.3 Navigation Flow

Screen	Description	Access
SplashScreen	App init, auth check, data sync trigger	All users
DataSyncScreen	Initial airport data download to SQLite	First launch
Login / Register	Authentication screens with role selection	Unauthenticated

BrokerProfileScreen	Complete broker account setup	New brokers
CourierProfile	Complete courier account setup	New couriers
DrawerScreen	Main navigation hub (sidebar drawer)	Authenticated
BrokerMap / CourierMap	Role-specific interactive map screen	Authenticated
BrokerMissions / CourierMissions	Mission list and management	Authenticated
EmptyLegMainScreen	Empty leg flight browsing	Couriers
ChatListScreen	Conversations list	Authenticated
NotificationsScreen	Push notification inbox	Authenticated
AdminPanel	User management and statistics	Admin only

4. User Roles & Actors

Broker Role

- ✓ Register and complete a broker profile (company name, website, country, license)
- ✓ Post job opportunities with departure/arrival details, bid amounts, and capacity
- ✓ Define milestones for each job with dates and expected completion criteria
- ✓ Search and view available couriers on an interactive map
- ✓ Filter couriers by location, availability, and rating
- ✓ Manage active missions: view status, update milestones, communicate with couriers
- ✓ Search and interact with empty aircraft legs
- ✓ Rate couriers upon mission completion
- ✓ Receive push notifications for courier responses and status updates
- ✓ Chat with couriers for mission coordination

Courier Role

- ✓ Register and complete a courier profile (personal details, passport, visa, license)
- ✓ Browse available jobs on an interactive map
- ✓ View job details including route, bid amount, and broker information
- ✓ Swipe through job cards to discover opportunities
- ✓ Browse empty aircraft legs and request to join
- ✓ Accept missions and track assigned milestones
- ✓ Mark milestones as complete upon delivery
- ✓ Rate brokers upon mission completion
- ✓ Receive push notifications for new jobs and updates
- ✓ Chat with brokers for mission coordination
- ✓ Update real-time GPS location for broker visibility

Admin Role

- ✓ Access the admin panel for user management
- ✓ View active and deleted user accounts
- ✓ Add, edit, or deactivate user accounts
- ✓ View user status charts and statistics
- ✓ Monitor platform activity

5. Functional Requirements — Authentication

FR-ID	Requirement	Priority	Notes
FR-AUTH-01	Users shall register with email and password	High	Firebase Auth
FR-AUTH-02	Users shall select a role (Broker or Courier) during registration	High	Role stored in Firestore
FR-AUTH-03	Users shall log in with email and password	High	Firebase Auth
FR-AUTH-04	App shall support 'Remember Me' functionality	Medium	SharedPreferences
FR-AUTH-05	New users shall complete profile setup after first login	High	Broker/Courier profile forms
FR-AUTH-06	App shall validate email format and password strength	High	Client-side validation
FR-AUTH-07	App shall display appropriate error messages on auth failure	High	User feedback
FR-AUTH-08	Users shall be able to log out from DrawerScreen	High	Clear local session
FR-AUTH-09	App shall navigate to correct dashboard based on user role	High	RoleProvider
FR-AUTH-10	App shall check authentication state on launch via SplashScreen	High	Auto-login if session valid
FR-AUTH-11	App shall support international phone number input	Medium	intl_phone_field package

5.1 Authentication Flow

Step 1	User opens app → SplashScreen checks Firebase auth state
Step 2	If not authenticated → Navigate to Login/Register screen
Step 3	User selects role (Broker/Courier) and enters credentials
Step 4	Firebase Auth validates credentials
Step 5	On success, check if profile is complete in Firestore
Step 6	If incomplete → Navigate to profile completion screen
Step 7	If complete → Navigate to DrawerScreen (main hub)
Step 8	RoleProvider sets user role and loads profile data

6. Functional Requirements — Broker Module

6.1 Broker Profile Management

FR-ID	Requirement	Priority	Notes
FR-BRK-01	Broker shall complete profile with company name, website, and country	High	Firestore storage
FR-BRK-02	Broker shall upload a profile picture	Medium	Firebase Storage
FR-BRK-03	Broker shall specify license/certification details	High	Compliance requirement
FR-BRK-04	Broker shall specify payment terms	High	Job postings reference this
FR-BRK-05	Broker shall view and edit their profile at any time	Medium	BrokerProfileScreen
FR-BRK-06	Broker profile shall display a star rating calculated from received reviews	High	Courier feedback

6.2 Job Posting

FR-ID	Requirement	Priority	Notes
FR-BRK-07	Broker shall create a new job posting via PlaceNewJob screen	High	Core feature
FR-BRK-08	Job posting shall include departure and arrival airports (IATA code supported)	High	AirportService integration
FR-BRK-09	Job posting shall include departure date, arrival date, and time	High	DateTimePicker
FR-BRK-10	Broker shall specify bid amount for the job	High	Payment field
FR-BRK-11	Broker shall specify number of couriers required	High	Capacity field
FR-BRK-12	Broker shall add multiple milestones to a job	High	AddNewMilestone screen
FR-BRK-13	Each milestone shall have a title, date, and description	High	MilestoneInputForm
FR-BRK-14	Posted jobs shall be stored in Firestore and visible to couriers on map	High	Real-time update
FR-BRK-15	Broker shall manage (view/update/delete) posted jobs via BrokerMissions screen	High	Mission management

6.3 Courier Discovery & Search

FR-ID	Requirement	Priority	Notes
FR-BRK-16	Broker shall view available couriers on an interactive Google Map	High	BrokerMap.dart
FR-BRK-17	Courier markers shall be clustered when zoomed out	Medium	cluster_manager package
FR-BRK-18	Broker shall search for couriers by name or criteria (SearchCourier)	High	Filter functionality
FR-BRK-19	Broker shall view courier profile details by tapping map marker	High	CourierClusterItem
FR-BRK-20	Broker shall see real-time courier locations updated via Firestore	High	Realtime Database
FR-BRK-21	Map shall show a radar animation while searching for couriers	High	RadarAnimation widget

6.4 Empty Leg Management

FR-ID	Requirement	Priority	Notes
FR-BRK-22	Broker shall search for available empty aircraft legs	High	SearchEmptyLeg screen

FR-BRK-23	Broker shall add new empty leg opportunities with route and flight details	High	AddNewMilestoneEmptyLeg
FR-BRK-24	Empty legs shall auto-expire and be cleaned up by WorkManager daily	Medium	Background task

6.5 Mission Overview & Analytics

FR-ID	Requirement	Priority	Notes
FR-BRK-25	Broker shall view all missions filtered by status (Pending/To Do/InProgress/Done)	High	See Missions screen
FR-BRK-26	App shall display mission progress with charts (bar chart, pie chart)	Medium	fl_chart, pie_chart
FR-BRK-27	Broker shall view detailed task information via TaskDetails screen	High	Task model data
FR-BRK-28	Broker shall manage legs and milestones per mission	High	ManageLegsAndMilestones
FR-BRK-29	Broker shall view and manage passport/visa documents	Medium	Passports.dart, Visas.dart

7. Functional Requirements — Courier Module

7.1 Courier Profile Management

FR-ID	Requirement	Priority	Notes
FR-COR-01	Courier shall complete profile with personal details	High	CourierProfile screen
FR-COR-02	Courier shall upload a profile picture	Medium	Firebase Storage
FR-COR-03	Courier shall add passport details (number, expiry, country)	High	Compliance requirement
FR-COR-04	Courier shall add visa information for relevant countries	High	Travel documentation
FR-COR-05	Courier shall add driver's license or other qualifications	Medium	Profile data
FR-COR-06	Courier shall specify payment terms and bank details	High	Payment processing
FR-COR-07	Courier profile shall display a star rating from broker reviews	High	Rating system
FR-COR-08	Courier shall update real-time GPS location for broker visibility	High	Geolocator + Firebase

7.2 Job Discovery

FR-ID	Requirement	Priority	Notes
FR-COR-09	Courier shall view available jobs on a Google Map	High	CourierMap.dart
FR-COR-10	Courier shall browse jobs via swipeable card stack (JobCard and StackWidget)	High	Card stack UI
FR-COR-11	Job cards shall display route, bid amount, dates, and broker info	High	JobDetails model
FR-COR-12	Courier shall view full job details by selecting a job (JobDetails screen)	High	Task detail view
FR-COR-13	Courier shall view broker profile information before accepting a job	High	BrokerBasicInfo screen
FR-COR-14	App shall show available jobs in real-time as brokers post	High	Firestore listener

7.3 Empty Leg Discovery

FR-ID	Requirement	Priority	Notes
FR-COR-15	Courier shall browse empty aircraft legs via EmptyLegMainScreen	High	Card stack browsing
FR-COR-16	Courier shall view flight details (route, date, operator)	High	FlightDetailsCard
FR-COR-17	Courier shall add new empty leg requests (AddEmptyLegDialog)	High	IATA code input
FR-COR-18	Courier shall request to join an empty leg flight	High	EmptyLegRequest model
FR-COR-19	Empty leg cards shall support swipe gesture for browsing	Medium	card_stack_widget

7.4 Mission & Milestone Management

FR-ID	Requirement	Priority	Notes
FR-COR-20	Courier shall view all assigned missions via CourierMissionScreen	High	Mission list
FR-COR-21	Courier shall view mission details including route and milestones	High	ViewCourierMission
FR-COR-22	Courier shall view all milestones for a specific mission	High	MilestonesScreen

FR-COR-23	Courier shall mark a milestone as complete (Complete Milestone screen)	High	Status update to Firestore
FR-COR-24	Milestone completion shall update status in real-time for higher visibility	High	Firestore write
FR-COR-25	Courier shall filter missions by status (Active, Completed, Pending)	Medium	Filter UI
FR-COR-26	Courier shall view task completion history	Medium	Completed tasks list

8. Functional Requirements — Chat & Communication

FR-ID	Requirement	Priority	Notes
FR-CHT-01	Users shall view a list of all active conversations (ChatListScreen)	High	Firestore chat collection
FR-CHT-02	Users shall open a specific conversation to send/receive messages	High	ChatDetailScreen
FR-CHT-03	Messages shall be delivered in real-time using Firestore listeners	High	Real-time updates
FR-CHT-04	Messages shall display sender name, timestamp, and read status	High	ChatBubble widget
FR-CHT-05	Users shall send text messages in a conversation	High	Core messaging
FR-CHT-06	Users shall attach and send files/documents (AttachmentButton)	High	Firebase Storage upload
FR-CHT-07	Users shall send and receive image messages	High	Image picker integration
FR-CHT-08	Users shall send and play video messages (VideoPlayerWidget)	High	Video player integration
FR-CHT-09	Chat conversations shall persist in Firestore across sessions	High	Permanent message history
FR-CHT-10	Users shall view chat partner's profile picture and name	Medium	profileScreen.dart
FR-CHT-11	Unread message count shall be displayed on ChatListScreen	Medium	Badge notification
FR-CHT-12	New message notifications shall be sent via FCM	High	NotificationService

8.1 Calling Feature

The application includes a **CallingScreen** for in-app voice/video calling functionality. This feature provides direct communication between brokers and couriers without leaving the application environment.

9. Functional Requirements — Notifications

FR-ID	Requirement	Priority	Notes
FR-NOT-01	App shall receive push notifications via Firebase Cloud Messaging	High	FCM integration
FR-NOT-02	Notifications shall work in foreground, background, and terminated states	High	All app states
FR-NOT-03	Tapping a notification shall navigate to the relevant screen	High	Deep linking via payload
FR-NOT-04	App shall display local notifications when FCM message arrives in foreground	High	flutter_local_notifications
FR-NOT-05	Notification payloads shall include type, target ID, and routing data	High	JSON payload structure
FR-NOT-06	Users shall view a notification inbox (NotificationsScreen)	High	Notification history
FR-NOT-07	Users shall view notification details (NotificationDetailScreen)	Medium	Detail view
FR-NOT-08	App shall send notifications to other users via FCM server	High	mopogotechnologies.com/fcm-server
FR-NOT-09	Notification types shall include: New Job, Milestone Update, Chat Message, Email	High	Notification types
FR-NOT-10	Broker shall receive notification when courier requests a job	High	Job request flow
FR-NOT-11	Courier shall receive notification when broker posts a matching job	Medium	Job matching

9.1 Notification States

State	Behavior	Handler
Foreground	Display local notification overlay	NotificationService.onMessage
Background	Show system notification; tap navigates to screen	FCM background handler
Terminated	App launched from notification; navigates on startup	getInitialMessage handler

10. Functional Requirements — Maps & Location

FR-ID	Requirement	Priority	Notes
FR-MAP-01	App shall display an interactive Google Map on Broker and Courier dashboards	High	google_maps_flutter
FR-MAP-02	Broker map shall show real-time courier locations as markers	High	Firebase Realtime DB
FR-MAP-03	Courier map shall show available job locations as markers	High	Firestore jobs collection
FR-MAP-04	Map markers shall be clustered in dense areas	Medium	google_maps_cluster_manager
FR-MAP-05	App shall request and handle GPS location permissions	High	permission_handler
FR-MAP-06	App shall track courier GPS location in real-time via Geolocator	High	Geolocator package
FR-MAP-07	Courier location shall be uploaded to Firebase Realtime Database periodically	High	location updates
FR-MAP-08	App shall support address-to-coordinate geocoding for airports	High	Geocoding package
FR-MAP-09	Airport search shall be available from local SQLite database	High	AirportService + SQLite
FR-MAP-10	App shall handle location permission denial gracefully with a message	High	PermissionsScreen
FR-MAP-11	Map shall display custom styled markers for different user types	Medium	Custom marker icons

10.1 Airport Database

The app includes a comprehensive airport database with **10,000+ airport records** sourced from a GitHub-hosted JSON file and cached locally in an SQLite database. The database is populated on first launch via **DataSyncScreen**.

Field	Type	Description
IATA Code	String	3-letter airport identifier (e.g., JFK, LHR)
GPS Code	String	ICAO/GPS identifier
Airport Name	String	Full official name
Country	String	Country of location
Latitude	Double	Geographic latitude coordinate
Longitude	Double	Geographic longitude coordinate
Elevation	Integer	Elevation in feet above sea level

11. Functional Requirements — Admin Panel

FR-ID	Requirement	Priority	Notes
FR-ADM-01	Admin shall access a user management screen (UserManagementScreen)	High	Admin-only access
FR-ADM-02	Admin shall view a list of all active user accounts	High	ActiveUsersTable
FR-ADM-03	Admin shall view a list of deleted/deactivated accounts	High	DeletedUsersTable
FR-ADM-04	Admin shall add new user accounts via a form dialog (UserFormDialog)	High	Create accounts
FR-ADM-05	Admin shall edit existing user account details	High	Edit form
FR-ADM-06	Admin shall deactivate or delete user accounts	High	Account management
FR-ADM-07	Admin panel shall display user status statistics via charts (UserStatusChart)	Medium	Analytics view
FR-ADM-08	Admin actions shall be logged for audit purposes	Medium	Firestore audit log

12. Non-Functional Requirements

12.1 Performance

NFR-ID	Requirement	Metric
NFR-PER-01	App shall launch and reach the splash screen within acceptable time	< 3 seconds on mid-range device
NFR-PER-02	Map shall render and display couriers/jobs with minimal latency	< 2 seconds on 4G
NFR-PER-03	Chat messages shall be delivered in real-time	< 1 second end-to-end
NFR-PER-04	Push notifications shall be received promptly after Firebase event	< 5 seconds
NFR-PER-05	Airport search shall return results from local SQLite cache quickly	< 500ms
NFR-PER-06	Images shall load from cache without visible delay	Cached Network Image
NFR-PER-07	App shall handle 1,000+ concurrent active users without degradation	Firestore scalability

12.2 Security

- All user data shall be transmitted over HTTPS/TLS encrypted connections
- Firebase Authentication shall manage all user credentials — no plain-text passwords stored
- Firestore Security Rules shall enforce role-based data access control
- Firebase Storage Security Rules shall restrict file access to authorized users only
- Sensitive data (API keys, Firebase config) shall not be hardcoded in public-facing code
- User sessions shall expire based on Firebase Auth token lifetime
- Device-level security (biometric/PIN) shall be respected by the application
- Admin panel access shall be restricted to verified admin accounts only

12.3 Reliability & Availability

NFR-ID	Requirement	Target
NFR-REL-01	Firebase backend shall maintain high availability	99.9% uptime (Firebase SLA)
NFR-REL-02	App shall detect network connectivity loss and notify user	Connectivity Plus
NFR-REL-03	App shall gracefully handle offline state without crashing	Offline-aware UI
NFR-REL-04	Local SQLite data shall persist across app restarts	Persistent cache
NFR-REL-05	Background WorkManager tasks shall retry on failure	Automatic retry policy

12.4 Usability

- App shall support both Android (API 21+) and iOS (13.0+) platforms
- UI shall follow Material Design guidelines for consistency
- App shall provide shimmer loading animations for better perceived performance
- App shall display toast notifications for quick user feedback
- All forms shall include input validation with clear error messages

- App shall support confirmation dialogs for destructive actions
- Navigation shall be intuitive with a sidebar drawer as the primary hub
- App shall support international phone number formats
- Date and time pickers shall provide a native platform experience

12.5 Maintainability & Scalability

- Code shall be organized into feature-based modules (broker/, courier/, common/, chat/)
- State management shall use the Provider pattern for testability and separation of concerns
- Service layer (FirestoreService, AirportService) shall abstract all backend operations
- Firebase Firestore allows horizontal scaling without infrastructure changes
- WorkManager enables background task scheduling independent of UI layer
- SQLite local database enables offline capability and reduces unnecessary API calls

13. Data Models

Task (Job Posting)

Field	Type	Description
taskId	String	Unique Firestore document ID
brokerId	String	Reference to broker user ID
departureAirport	String	IATA code of departure
arrivalAirport	String	IATA code of arrival
departureDate	DateTime	Scheduled departure date/time
arrivalDate	DateTime	Scheduled arrival date/time
bidAmount	double	Payment offered for the job
courierCapacity	int	Number of couriers required
status	String	pending / todo / inProgress / completed
milestones	List<Milestone>	List of task milestones
assignedCourierId	String?	Courier assigned to this task
createdAt	Timestamp	Job creation timestamp

Milestone

Field	Type	Description
milestoneId	String	Unique milestone identifier
title	String	Milestone name/title
description	String	Detailed description
dueDate	DateTime	Expected completion date
status	String	pending / inProgress / completed
completedAt	DateTime?	Actual completion timestamp
taskId	String	Parent task reference

BrokerProfileData

Field	Type	Description
uid	String	Firebase Auth user ID
companyName	String	Broker company name
website	String	Company website URL
country	String	Country of operation
licenseNumber	String	Broker license/certification
paymentTerms	String	Payment terms description

profilePictureUrl	String?	Firebase Storage URL
rating	double	Average rating (0.0 - 5.0)
totalRatings	int	Number of ratings received
fcmToken	String	FCM device token for notifications

CourierProfileData

Field	Type	Description
uid	String	Firebase Auth user ID
firstName	String	Courier first name
lastName	String	Courier last name
email	String	Contact email
phoneNumber	String	International format phone
passports	List<Passport>	Travel documents
visas	List<Visa>	Visa records
paymentTerms	String	Payment preference
profilePictureUrl	String?	Firebase Storage URL
rating	double	Average rating (0.0 - 5.0)
fcmToken	String	FCM device token for notifications
latitude	double?	Real-time GPS latitude
longitude	double?	Real-time GPS longitude
isOnline	bool	Online availability status

EmptyLegRequest

Field	Type	Description
id	String	Unique document ID
departureAirport	String	IATA departure code
arrivalAirport	String	IATA arrival code
flightDate	DateTime	Available flight date
operatorId	String	Creator user ID
seats	int	Available seats
expiresAt	DateTime	Auto-expiration timestamp

Rating

Field	Type	Description
ratingId	String	Unique rating ID
fromUserId	String	User who gave the rating
toUserId	String	User who received the rating

taskId	String	Related task/job reference
stars	double	Star rating (1.0 - 5.0)
comment	String?	Optional review comment
createdAt	Timestamp	Rating submission timestamp

14. External Interfaces

14.1 Firebase Services

Service	Usage	Configuration
Firebase Auth	User registration, login, session management	Email/Password provider enabled
Cloud Firestore	Jobs, users, chat, ratings, notifications storage	Project: bf-obc-flutter
Firebase Realtime Database	Real-time courier location updates	Real-time sync
Firebase Storage	Profile pictures, documents, media files	Security rules restricted
Firebase Messaging (FCM)	Push notifications to devices	iOS & Android configured

14.2 Google APIs

API	Usage	Package
Google Maps JavaScript API	Render interactive maps with markers and clustering	google_maps_flutter v2.3.0
Geocoding API	Convert addresses to coordinates and vice versa	geocoding v2.1.0
Geolocation API	Access device GPS coordinates	geolocator v10.1.0

14.3 Custom Backend API

The application interfaces with a custom FCM notification server for sending push notifications between users:

Endpoint	Method	Purpose
https://mopogotechnologies.com/fcm-server/send_notification.php	POST	Send FCM push notification to target device

14.4 GitHub Airport Data Source

Airport data is fetched from a GitHub-hosted JSON file containing 10,000+ airport records. This data is downloaded on first app launch and stored in a local SQLite database, eliminating the need for repeated network requests.

15. Security Requirements

SEC-ID	Requirement	Priority	Implementation
SEC-01	All network communication shall use HTTPS/TLS encryption	High	Firebase + Dio default HTTPS
SEC-02	User passwords shall never be stored in plain text	High	Firebase Auth hashing
SEC-03	Firebase Auth tokens shall be used for all authenticated API calls	High	Firebase SDK auth headers
SEC-04	Firestore Security Rules shall enforce that users can only read/write their own data	High	Firestore Rules config
SEC-05	Firebase Storage rules shall restrict file access to authenticated users	High	Storage Rules config
SEC-06	FCM tokens shall be refreshed and updated in Firestore on each app launch	High	Token refresh logic
SEC-07	Admin panel features shall be restricted to users with admin role in Firestore	High	Firestore role checks
SEC-08	App shall not log sensitive user data (passwords, tokens) to console	High	Build configuration
SEC-09	Google API keys shall be restricted to specific app bundle IDs and domains	High	Google API restrictions
SEC-10	File uploads shall be validated for type and size before sending to Firebase Storage	Medium	File upload validation

16. Constraints & Assumptions

16.1 Technical Constraints

- Flutter SDK version $\geq 3.1.5$ is required for development
- Firebase project (bf-obc-flutter) must remain active for all backend services
- Google Maps API key must have Maps SDK enabled for Android and iOS
- FCM server (mopogotechnologies.com) must be operational for push notifications
- Android minimum SDK: API 21 (Android 5.0 Lollipop)
- iOS minimum deployment target: iOS 13.0
- Internet connection is required for all real-time features (maps, chat, jobs)
- GPS permission must be granted for location-based features to function
- Camera/Gallery permission required for profile picture and media uploads
- WorkManager background tasks require battery optimization to be disabled on some devices

16.2 Business Constraints

- The platform targets the aviation courier/logistics sector specifically
- Users must have a valid email address to register
- Brokers must complete their company profile before posting jobs
- Couriers must complete travel documentation (passport/visa) to be discoverable
- Payment processing is handled externally — the app specifies terms only
- Empty leg data expires automatically and is not archived

16.3 Assumptions

- All users have a compatible smartphone (Android 5.0+ or iOS 13.0+)
- Users have stable internet access for real-time features
- Firebase quotas are sufficient for the expected user load
- Google Maps API usage stays within quota limits
- The GitHub-hosted airport JSON file remains accessible and formatted consistently
- Admin users are designated manually via Firestore user roles
- The FCM server handles notification delivery reliably
- Users consent to location tracking while using the app
- Currency handling for bid amounts is managed by brokers and couriers directly

17. Glossary

Term	Definition
Broker	A company or individual who creates and posts courier job opportunities on the platform
Courier	A professional who browses, accepts, and fulfills logistics/courier job assignments
Mission	A complete job posting that may contain multiple legs and milestones
Milestone	A specific deliverable step within a mission with its own status and due date
Empty Leg	An available aircraft journey with empty cargo/passenger capacity offered to couriers
FCM	Firebase Cloud Messaging — Google's cross-platform push notification service
IATA Code	3-letter airport code assigned by the International Air Transport Association
ICAO/GPS Code	4-letter airport identifier used in flight planning and GPS systems
Firestore	Google Cloud Firestore — a NoSQL cloud database part of Firebase
Provider	Flutter's recommended state management solution using InheritedWidget pattern
WorkManager	Android library for guaranteed background task execution
SQLite	Lightweight, file-based relational database used for local data caching
Geolocator	Flutter plugin for accessing device GPS location data
SRS	Software Requirements Specification — this document
MVP	Minimum Viable Product — the initial release with core features only
API	Application Programming Interface — contract for software component communication
FCM Token	Unique device identifier used to target push notifications
Shimmer	Loading placeholder animation that mimics content layout while data loads
Cluster	Grouped map markers that represent multiple items at the same zoom level